

SIMATSENGINEERING

ASSESSMENT TOOL 3: Modelling Comparative Analysis Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages. (BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Modelling Comparative Analysis Assignment

Code of Conduct:

I Kamalli shree V(192525030) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question/Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities of each approach.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				
Ease of Implementation				
Suitability for the Problem				

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10%–Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8%–Good understanding with minor omissions.	5–6%–Basic understanding with limited analysis.	0–4%–Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8%–Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15%–Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12%–Rules are appropriate with minor errors or omissions.	7–9%–Basic rules are developed but several are incomplete or unclear.	0–6%–Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8%–Good propositional representation with minor errors.	5–6%–Basic representation with limited logical formulation.	0–4%–Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15%–Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12%–Good FOL model with minor errors or omissions.	7–9%–Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6%–FOL model is largely incorrect or missing.

6. Prolog Modelling and Implementation	15%	13–15%– Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12%– Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6%– Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8%– Both mechanisms are demonstrated with minor gaps.	5–6%– Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10%– Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8%– Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4%– Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2%– Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0%– No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5%– Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				

CYBERSECURITY THREAT DETECTION EXPERT SYSTEM

1. Problem Statement & Objectives

1.1 Problem Statement

A Security Operations Center (SOC) needs to identify potential cybersecurity threats from suspicious user and system activities. Common indicators include repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic.

The objective is to develop a rule-based cybersecurity threat detection model that represents these security events using production rules, propositional logic, first-order logic, and Prolog. The system should use logical inference to identify possible threats and recommend appropriate security actions.

1.2 Objectives

The main objectives are:

- To model cybersecurity events and threats using logical knowledge representation.
- To represent security knowledge using IF–THEN production rules.
- To express the same knowledge using propositional logic.
- To represent relationships using first-order logic.
- To implement the threat-detection system using SWI-Prolog.
- To demonstrate forward and backward chaining.
- To demonstrate Prolog concepts such as unification and backtracking.
- To evaluate the different reasoning models and identify the most suitable approach

1.3 Input

- The system accepts security events such as:
- `repeated_login_failures`
- `unusual_login_location`

- privilege_escalation
- suspicious_file_access
- abnormal_network_traffic

1.4 Output

The system identifies probable cybersecurity threats such as:

- brute_force_attack
- account_compromise
- privilege_abuse
- data_exfiltration
- network_intrusion
- advanced_account_threat

It also provides recommended security actions.

2. Domain Knowledge & Facts

2.1 Domain Entities

Entity	Description
User	Person accessing a system
Security Event	Suspicious activity observed by the system

Threat	Possible cybersecurity attack or security violation
Security Action	Recommended response to a detected threat
Rule	Relationship connecting security events to threats

2.2 Security Events

The system considers five major security events:

1. Repeated login failures
2. Unusual login location
3. Privilege escalation
4. Suspicious file access
5. Abnormal network traffic

2.3 Possible Threats

Security Events	Possible Threat
Repeated login failures	Brute-force attack
Repeated login failures + unusual location	Account compromise

Privilege escalation	Privilege abuse
Suspicious file access + abnormal network traffic	Data exfiltration
Abnormal network traffic	Network intrusion
Login failures + unusual location + privilege escalation	Advanced account threat

2.4 Security Actions

The system can recommend actions such as:

- Block the account
- Verify user identity
- Review user privileges
- Isolate the affected system
- Inspect network activity
- Review file-access logs
- Enable additional authentication

3. Production Rule Model

The domain knowledge is represented using IF–THEN production rules.

Example 1

IF repeated_login_failures
THEN brute_force_attack

Example 2

IF repeated_login_failures
AND unusual_login_location
THEN account_compromise

Example 3

IF privilege_escalation
THEN privilege_abuse

Example 4

IF suspicious_file_access
AND abnormal_network_traffic
THEN data_exfiltration

Example 5

IF abnormal_network_traffic
THEN network_intrusion

Example 6

IF repeated_login_failures
AND unusual_login_location
AND privilege_escalation
THEN advanced_account_threat

The production rules allow the system to transform observed security events into probable cybersecurity threats.

```
event(repeated_login_failures).  
event(unusual_login_location).  
event(privilege_escalation).  
event(suspicious_file_access).  
event(abnormal_network_traffic).  
  
rule(brute_force_attack,  
    [repeated_login_failures],  
    [block_account,  
enable_additional_authentication]).  
  
rule(account_compromise,  
    [repeated_login_failures, |  
unusual_login_location],  
    [verify_user_identity, block_account]).
```

4. Propositional Logic Model

In propositional logic, each security event is represented by a proposition that can be either true or false.

Let:

- L = repeated login failures
- U = unusual login location
- P = privilege escalation
- F = suspicious file access
- N = abnormal network traffic

Threat propositions:

- B = brute-force attack
- C = account compromise
- A = privilege abuse
- D = data exfiltration
- I = network intrusion

4.1 Logical Statements

Brute-force attack

- $L \rightarrow B$

Meaning:

If repeated login failures occur, a brute-force attack may be indicated.

Account compromise

- $(L \wedge U) \rightarrow C$

Privilege abuse

- $P \rightarrow A$

Data exfiltration

- $(F \wedge N) \rightarrow D$

Network intrusion

- $N \rightarrow I$

Example inference

Given:

- $L = \text{True}$
- $U = \text{True}$
- $(L \wedge U) \rightarrow C$

we can derive:

- $C = \text{True}$

Therefore, an account compromise is detected by the propositional model.

5. First-Order Logic Model

First-order logic provides a richer representation because it can represent entities, relationships, properties, and quantified statements.

Let:

- $\text{User}(x)$
- $\text{Event}(x,e)$

- Threat(x,t)

represent that a user, event, and threat are associated with an entity.

Example predicates

- LoginFailure(user)
- UnusualLocation(user)
- PrivilegeEscalation(user)
- SuspiciousFileAccess(user)
- AbnormalNetworkTraffic(user)

Rules:

Brute-force attack

- $\forall x [\text{LoginFailure}(x) \rightarrow \text{BruteForceAttack}(x)]$

Account compromise

- $\forall x [(\text{LoginFailure}(x) \wedge \text{UnusualLocation}(x)) \rightarrow \text{AccountCompromise}(x)]$

Privilege abuse

- $\forall x [\text{PrivilegeEscalation}(x) \rightarrow \text{PrivilegeAbuse}(x)]$

Data exfiltration

- $\forall x [(SuspiciousFileAccess(x) \wedge AbnormalNetworkTraffic(x)) \rightarrow DataExfiltration(x)]$

Network intrusion

- $\forall x [AbnormalNetworkTraffic(x) \rightarrow NetworkIntrusion(x)]$

The use of variables and quantifiers allows the same rule to apply to multiple users or systems.

6. Prolog Model & Implementation

The system was implemented using SWI-Prolog in a single source file: cybersecurity_expert.pl

6.1 Prolog Facts

- event(repeated_login_failures).
- event(unusual_login_location).
- event(privilege_escalation).
- event(suspicious_file_access).
- event(abnormal_network_traffic).

6.2 Prolog Rules

For example:

```
rule(account_compromise,  
      [repeated_login_failures, unusual_login_location],  
      [verify_user_identity, block_account]).
```

The first argument represents the threat, the second contains the required security events, and the third contains recommended security actions.

6.3 Main Predicates

Predicate	Function
add_event/1	Adds a security event
clear_data/0	Clears stored events
diagnose/1	Checks a suspected threat
forward_chain/0	Performs forward reasoning
show_threats/0	Displays detected threats
recommend/1	Displays recommended actions
unification_demo/0	Demonstrates unification
backtracking_demo/0	Demonstrates backtracking

run_tests/0	Executes test cases
start/0	Starts the user interface

```

=====
                        SECURITY EVENT INPUT
=====
Available security events:
1. repeated_login_failures
2. unusual_login_location
3. privilege_escalation
4. suspicious_file_access
5. abnormal_network_traffic

Enter number of events: |: 2.
Enter security event: |: repeated_login_failures.
Enter security event: |: unusual_login_location.

Known Security Events
-----
-> repeated_login_failures
-> unusual_login_location

=====

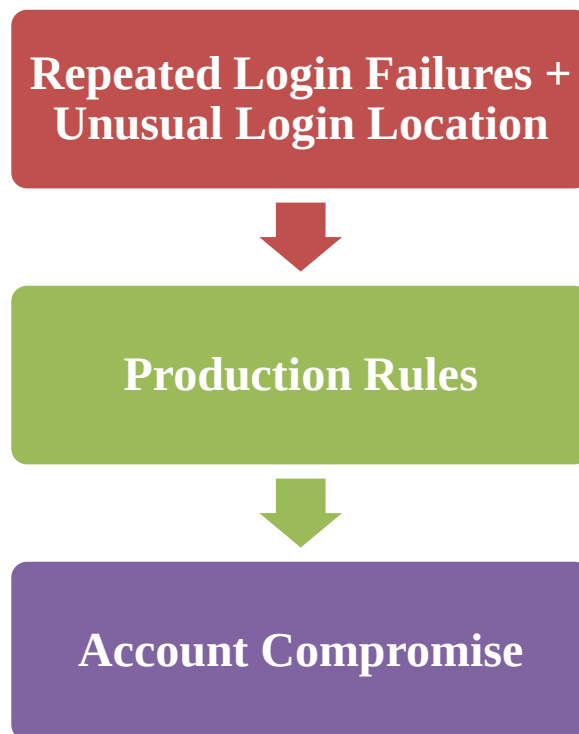
```

7. Forward & Backward Chaining

7.1 Forward Chaining

Forward chaining is a data-driven inference mechanism. It starts with known security events and applies matching production rules.

For example:



```

=====
CYBERSECURITY THREAT DETECTION SYSTEM
=====
Rule-Based Security Expert System using Prolog
-----
1. Enter security events
2. Forward chaining
3. Backward chaining
4. Show detected threats
5. Show recommendations
6. Unification demonstration
7. Backtracking demonstration
8. Run test cases
9. Clear security data
10. Exit
-----
Enter option: |: 2.

```

```

=====
FORWARD CHAINING
=====
Starting from known security events...
[Derived] brute_force_attack
[Derived] account_compromise

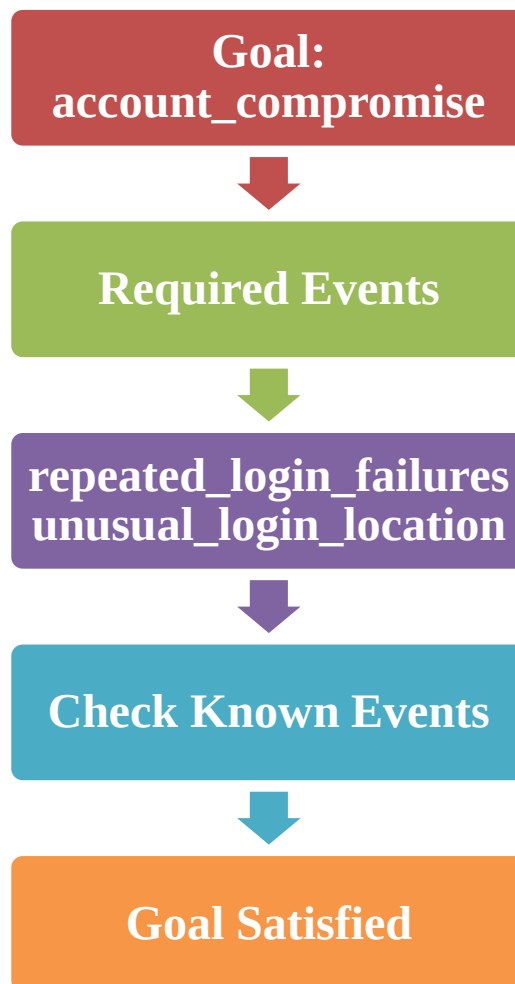
Forward chaining completed.

```

7.2 Backward Chaining

Backward chaining is goal-driven inference. It begins with a suspected threat and checks whether the required security events are known.

For example:



Enter option: |: 3.
Enter suspected threat: |: account_compromise.

=====

BACKWARD CHAINING

=====

Goal: account_compromise

Result: Goal satisfied.
account_compromise is supported by the known security events.

8. Comparative Analysis.

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation	IF-THEN rules	True/false propositions	Predicates, variables and quantifiers	Facts, predicates and rules
Expressiveness	Moderate	Limited	High	High
Rule Representation	Direct and intuitive	Logical implications	Predicate-based rules	Natural rule syntax
Inference Mechanism	Rule matching	Logical inference	Predicate inference	Automated logical inference
Forward Chaining	Possible	Possible	Possible	Easily implement

				ed
Backward Chaining	Possible	Possible	Possible	Naturally supported
Handling Relationships	Limited	Weak	Strong	Strong
Scalability	Moderate	Moderate	High	High for rule-based systems
Explainability	High	High	High	High
Ease of Implementation	Easy	Moderate	More complex	Easy for rule-based reasoning
Suitability for Problem	High	Moderate	High	Very High

Comparison Discussion

Production Rules

Production rules are highly suitable for representing direct security relationships such as:

IF repeated login failures
THEN brute-force attack

Propositional Logic

Propositional logic is useful when security events can simply be represented as true or false. However, it becomes less expressive when relationships between users, systems, events, and threats need to be represented.

First-Order Logic

First-order logic provides greater expressiveness through variables, predicates, and quantifiers. It can represent relationships involving multiple users and systems more effectively.

Prolog

Prolog is particularly suitable because it combines facts, rules, unification, backtracking, and logical inference in a single programming environment. It can naturally support both forward and backward reasoning in this project.

9. Results, Discussion & Recommended Model

9.1 Results

The system was tested using five cybersecurity scenarios.

Test Case	Input Security Events	Expected Threat
TC1	Repeated login failures	Brute-force attack
TC2	Login failures + unusual location	Account compromise + brute-force attack
TC3	Privilege escalation	Privilege abuse
TC4	Suspicious file access + abnormal network traffic	Data exfiltration + network intrusion
TC5	Login failures + unusual location + privilege escalation	Multiple account threats

The implemented rules successfully map the defined security events to the corresponding threats in the knowledge base.

=====

TEST CASE 1: Repeated Login Failures

=====

=====

SECURITY THREAT ANALYSIS

=====

Known Security Events

-> repeated_login_failures

Applying production rules...
[Derived] brute_force_attack

Detected Cybersecurity Threats

-> brute_force_attack

Explanation for: brute_force_attack
Required events: [repeated_login_failures]
Recommended actions:
-> block_account
-> enable_additional_authentication

=====

TEST CASE 3: Privilege Escalation

=====

=====

SECURITY THREAT ANALYSIS

=====

Known Security Events

-> privilege_escalation

Applying production rules...
[Derived] privilege_abuse

Detected Cybersecurity Threats

-> privilege_abuse

Explanation for: privilege_abuse
Required events: [privilege_escalation]
Recommended actions:
-> review_user_privileges
-> review_file_access_logs

=====

TEST CASE 2: Login Failures + Unusual Location

=====

=====

SECURITY THREAT ANALYSIS

=====

Known Security Events

-> repeated_login_failures
-> unusual_login_location

Applying production rules...
[Derived] brute_force_attack
[Derived] account_compromise

Detected Cybersecurity Threats

-> brute_force_attack
-> account_compromise

Explanation for: brute_force_attack
Required events: [repeated_login_failures]
Recommended actions:
-> block_account
-> enable_additional_authentication

Explanation for: account_compromise
Required events: [repeated_login_failures,unusual_login_location]
Recommended actions:
-> verify_user_identity
-> block_account

=====

TEST CASE 4: Suspicious File Access + Network Traffic

=====

=====

SECURITY THREAT ANALYSIS

=====

Known Security Events

-> suspicious_file_access
-> abnormal_network_traffic

Applying production rules...
[Derived] data_exfiltration
[Derived] network_intrusion

Detected Cybersecurity Threats

-> data_exfiltration
-> network_intrusion

Explanation for: data_exfiltration
Required events: [suspicious_file_access,abnormal_network_traffic]
Recommended actions:
-> review_file_access_logs
-> inspect_network_activity

Explanation for: network_intrusion
Required events: [abnormal_network_traffic]
Recommended actions:
-> inspect_network_activity
-> isolate_affected_system

```
=====
TEST CASE 5: Multiple Account Threat Indicators
=====
```

```
=====
SECURITY THREAT ANALYSIS
=====
```

```
Known Security Events
```

```
-----
```

- > repeated_login_failures
- > unusual_login_location
- > privilege_escalation

```
Applying production rules...
```

- [Derived] brute_force_attack
- [Derived] account_compromise
- [Derived] privilege_abuse
- [Derived] advanced_account_threat

```
Detected Cybersecurity Threats
```

```
-----
```

- > brute_force_attack
- > account_compromise
- > privilege_abuse
- > advanced_account_threat

```
Explanation for: brute_force_attack
```

```
Required events: [repeated_login_failures]
```

```
Recommended actions:
```

- > block_account
- > enable_additional_authentication

```
Explanation for: account_compromise
```

```
Required events: [repeated_login_failures,unusual_login_location]
```

```
Recommended actions:
```

- > verify_user_identity
- > block_account

9.2 Discussion

The results demonstrate that rule-based reasoning can be used to model initial cybersecurity threat detection. Different combinations of events can produce different threats, and a single security event can contribute to more than one possible conclusion.

For example, repeated login failures can indicate a brute-force attack, while the combination of repeated login failures and an unusual login location provides stronger evidence for a possible account compromise.

9.3 Recommended Model

Prolog is recommended for this problem.

The main reasons are:

- It naturally represents facts and rules.
- It supports logical inference.
- It provides unification and backtracking.
- It can perform goal-directed backward reasoning.
- Rules are relatively easy to modify and extend.
- The reasoning process can be explained clearly.

For a larger production SOC system, Prolog alone would not be sufficient; it would need to be integrated with real-time logs, SIEM systems, authentication systems, network telemetry, and other security infrastructure.

9.4 Limitations

The current system:

- Uses a small predefined knowledge base.
- Does not process live security logs.
- Does not connect to a real SIEM.
- Does not assign numerical threat probabilities.
- Does not process actual network packets.
- Uses simplified academic threat-detection rules.

9.5 Possible Improvements

Future versions could include:

- Real-time log ingestion
- SIEM integration
- User and device identity tracking
- IP address and geographic analysis
- Risk scoring

- Machine-learning-assisted threat detection
- Automated incident response
- Larger cybersecurity knowledge bases

10. Conclusion

The Cybersecurity Threat Detection Expert System was developed to model security events and identify probable threats using logical reasoning. The system represents cybersecurity knowledge through production rules and implements the reasoning process using Prolog.

The project also models the same domain using propositional logic and first-order logic, allowing the strengths and limitations of different knowledge-representation approaches to be compared. Forward chaining derives threats from known security events, while backward chaining verifies whether a suspected threat is supported by the available evidence.

The testing scenarios demonstrate that the system can identify the threats represented in its knowledge base and provide corresponding security recommendations. Among the approaches considered, Prolog is the most suitable for this

academic problem because it combines declarative knowledge representation with logical inference, unification, and backtracking.

The system is a simplified academic model. Integrating real-time security logs, SIEM platforms, threat intelligence, and automated response mechanisms would be required for practical cybersecurity deployment.

11. References

1. SWI-Prolog Documentation — Prolog language and reference documentation.
2. NIST Cybersecurity Framework — Cybersecurity risk management concepts.
3. NIST Computer Security Resource Center — Cybersecurity terminology and guidance.
4. MITRE ATT&CK — Knowledge base of adversary tactics and techniques.

12. GitHub

<https://github.com/kamallishree/MLA-0103/tree/main/CO5>